

1. `insert(int father, int data)`

- **Mô tả:** Tìm cha (`father`) và thêm Node mới (`data`) vào cây.
- **Phân tích độ phức tạp:**
 - Tìm cha bằng hàm đệ quy `findNode()`, độ phức tạp của việc tìm kiếm phụ thuộc vào số lượng Node trong cây.
 - Duyệt qua danh sách các con của Node cha để kiểm tra xem Node có trùng lặp không.
 - **Độ phức tạp:** Tìm cha và duyệt con đều phụ thuộc vào kích thước của cây, do đó **độ phức tạp là $O(n)$** , với n là số lượng Node trong cây.
 - **Trường hợp tốt nhất:** Cây có ít Node, tìm thấy cha ngay ở gốc. **Độ phức tạp là $O(1)$** .
 - **Trường hợp xấu nhất:** Phải duyệt hết cây để tìm Node cha. **Độ phức tạp là $O(n)$** .

2. `findNode(Node *node, int data)`

- **Mô tả:** Hàm đệ quy tìm kiếm Node có giá trị bằng `data`.
- **Độ phức tạp:** Trong trường hợp xấu nhất, phải duyệt qua toàn bộ cây để tìm Node có giá trị tương ứng.
 - **Độ phức tạp là $O(n)$** .
 - **Trường hợp tốt nhất:** Node cần tìm là gốc hoặc nằm gần gốc. **$O(1)$** .
 - **Trường hợp xấu nhất:** Node cần tìm nằm ở vị trí cuối cùng của cây. **$O(n)$** .

3. `getNumOfChild(Node *node)`

- **Mô tả:** Tính số lượng Node con của một Node, bao gồm cả các con cấp dưới (đệ quy).
- **Độ phức tạp:** Hàm đệ quy sẽ duyệt qua tất cả các Node con của `node`.
 - **Độ phức tạp là $O(n)$** .
 - **Trường hợp tốt nhất:** `node` không có con. **$O(1)$** .
 - **Trường hợp xấu nhất:** `node` có nhiều con cấp dưới. **$O(n)$** .

4. `remove(int data)`

- **Mô tả:** Xóa Node có giá trị `data` và các con của nó.
- **Độ phức tạp:** Hàm này phụ thuộc vào việc tìm Node cần xóa (`findNode()`) và việc đệ quy xóa toàn bộ con của nó (`getNumOfChild()`).
 - **Độ phức tạp là $O(n)$** .
 - **Trường hợp tốt nhất:** Node cần xóa là lá, chỉ cần tìm Node và xóa trực tiếp. **$O(1)$** .
 - **Trường hợp xấu nhất:** Node cần xóa có nhiều Node con. **$O(n)$** .

5. `preorder(Node *node)` và `postorder(Node *node)`

- **Mô tả:** Duyệt cây theo thứ tự trước (`preorder`) hoặc sau (`postorder`).

- **Độ phức tạp:** Mỗi Node được duyệt qua đúng 1 lần.
 - **Độ phức tạp là $O(n)$** cho cả hai hàm.

6. **isBinaryTree(Node *node)**

- **Mô tả:** Kiểm tra xem cây có phải là cây nhị phân không (mỗi Node có tối đa 2 con).
- **Độ phức tạp:** Hàm duyệt qua toàn bộ cây để kiểm tra số lượng con của mỗi Node.
 - **Độ phức tạp là $O(n)$.**

7. **isBinarySearchTree(Node *node)**

- **Mô tả:** Kiểm tra cây có phải cây tìm kiếm nhị phân (BST) không.
- **Độ phức tạp:** Hàm kiểm tra điều kiện của các Node và các con đệ quy. Mỗi Node được kiểm tra đúng 1 lần.
 - **Độ phức tạp là $O(n)$.**

8. **isMaxHeapTree(Node *node)**

- **Mô tả:** Kiểm tra xem cây có phải là Max Heap Tree không.
- **Độ phức tạp:** Hàm duyệt qua tất cả các Node và kiểm tra điều kiện với các con của nó.
 - **Độ phức tạp là $O(n)$.**

9. **inorder(Node *node)**

- **Mô tả:** Duyệt cây theo thứ tự giữa (**inorder**) (chỉ dành cho cây nhị phân).
- **Độ phức tạp:** Mỗi Node được duyệt qua đúng 1 lần.
 - **Độ phức tạp là $O(n)$.**

10. **height(Node *node)**

- **Mô tả:** Tính chiều cao của cây (số tầng lớn nhất).
- **Độ phức tạp:** Hàm đệ quy duyệt qua tất cả các Node và tính chiều cao của từng nhánh.
 - **Độ phức tạp là $O(n)$.**

11. **depth(int data)**

- **Mô tả:** Tính độ sâu của một Node (số tầng tính từ gốc).
- **Độ phức tạp:** Phụ thuộc vào việc tìm Node bằng **findNode()** và tính độ sâu qua các cha của nó.
 - **Độ phức tạp là $O(n)$.**

12. **numOfLeaves(Node *node)**

- **Mô tả:** Đếm số lượng Node lá.
- **Độ phức tạp:** Duyệt qua tất cả các Node để kiểm tra và đếm lá.
 - **Độ phức tạp là $O(n)$.**

13. `findMax(Node *root)`

- **Mô tả:** Tìm giá trị lớn nhất trong cây.
- **Độ phức tạp:** Hàm duyệt qua toàn bộ cây để so sánh giá trị của mỗi Node.
 - **Độ phức tạp là $O(n)$.**

14. `findMaxChild(Node *root)`

- **Mô tả:** Tìm Node có số con lớn nhất.
- **Độ phức tạp:** Hàm đệ quy duyệt qua tất cả các Node và kiểm tra số con của từng Node.
 - **Độ phức tạp là $O(n)$.**

Tổng kết:

- Hầu hết các hàm đều có **độ phức tạp $O(n)$** do phải duyệt qua toàn bộ cây, trong đó n là số lượng Node trong cây.
- Các trường hợp tốt nhất là khi tìm Node hoặc thao tác với cây có kích thước nhỏ hoặc Node cần xử lý nằm ở gần gốc.